The Journal of Open Source Software

HipSTR-MT: A Multi-threaded Extension of HipSTR for Faster Short Tandem Repeat Genotyping

Joachim Galil [1], **Melissa Gymrek** [2,3,4], **and Yatish Turakhia** [1¶]

1 Department of Electrical and Computer Engineering, University of California San Diego, San Diego, CA 92093, USA **2** Department of Medicine, University of California San Diego, San Diego, CA 92093, USA **3** Department of Pediatrics, University of California San Diego, San Diego, CA 92093, USA **4** Department of Computer Science & Engineering, University of California San Diego, San Diego, CA 92093, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 24 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Short tandem repeats (STRs) are highly variable, repetitive regions of the genome that serve as important markers in population genetics, forensics, and disease research (Budowle & Sajantila, 2024; Uguen et al., 2024). HipSTR (Willems et al., 2017) is a widely used tool for genotyping STRs from whole-genome sequencing data, using a specialized hidden Markov model to align reads to candidate alleles while accounting for PCR stutter artifacts and phased SNP haplotypes. However, its serial execution model limits throughput on modern multi-core hardware, particularly in large cohort studies genotyping hundreds of thousands of loci across many samples. HipSTR-MT restructures HipSTR's core loop into a three-stage concurrent pipeline built on the Taskflow task-graph framework (Huang et al., 2022), preserving HipSTR's full feature set while producing genotype-identical output. On the NA12891 sample dataset, HipSTR-MT achieves a 32.5× reduction in wall-clock runtime at 64 threads relative to the serial baseline, letting researchers use multi-core workstations and HPC nodes without changing existing HipSTR workflows.

Statement of need

Genome-wide STR genotyping is computationally intensive: for each locus, HipSTR seeks and filters BAM/CRAM records, constructs candidate haplotypes via a de Bruijn graph, aligns every read against every candidate haplotype under a stutter emission model, and estimates genotype posteriors—all on a single thread—making genome-wide analysis of large cohorts impractical on one machine. Cluster-based parallelism (running separate samples as independent jobs) is a common workaround, but it does not reduce the per-sample latency of genotyping a single large sample or a targeted large-region analysis, and requires infrastructure not all users have. HipSTR-MT closes this gap with intra-run thread-level parallelism, giving users of standard multi-core workstations a drop-in performance improvement via a single added `--threads` flag. It is intended for population geneticists, forensic genomics researchers, and any group running HipSTR as part of a larger cohort-scale variant-calling pipeline.

State of the Field

HipSTR is one of several tools for genome-wide STR genotyping from short-read data, alongside GangSTR and ExpansionHunter, which additionally use mate-pair distance to detect STR expansions longer than a single read (Oketch et al., 2024). Independent benchmarking shows GangSTR and ExpansionHunter can outperform HipSTR on call rate and memory usage, but HipSTR remains competitive in genotyping accuracy for common STR loci, including the

DRAFT

CODIS core STRs used in forensics (Oketch et al., 2024). Additionally, a unique advantage of HipSTR is that it infers sequence in addition to length variation and models haplotypes jointly across samples. None of these tools natively parallelizes within a single sample; throughput on large cohorts is instead typically achieved through cluster-based job scheduling across many machines (Ahmad et al., 2022; Mirchandani et al., 2024), which increases aggregate throughput but not per-sample latency, and assumes multi-node HPC access. HipSTR-MT takes a complementary approach: intra-run thread-level parallelism so a single workstation can genotype a sample or region substantially faster, while leaving HipSTR's statistical model and output format unchanged. HipSTR-MT is developed as a fork because these changes restructure HipSTR's core processing loop and add build requirements (C++20, plus vendored Taskflow, mimalloc, libdeflate, and an upgraded htslib); now that they are validated as genotype-identical to upstream, we plan to contribute them to the maintained gymrek-lab version of HipSTR.

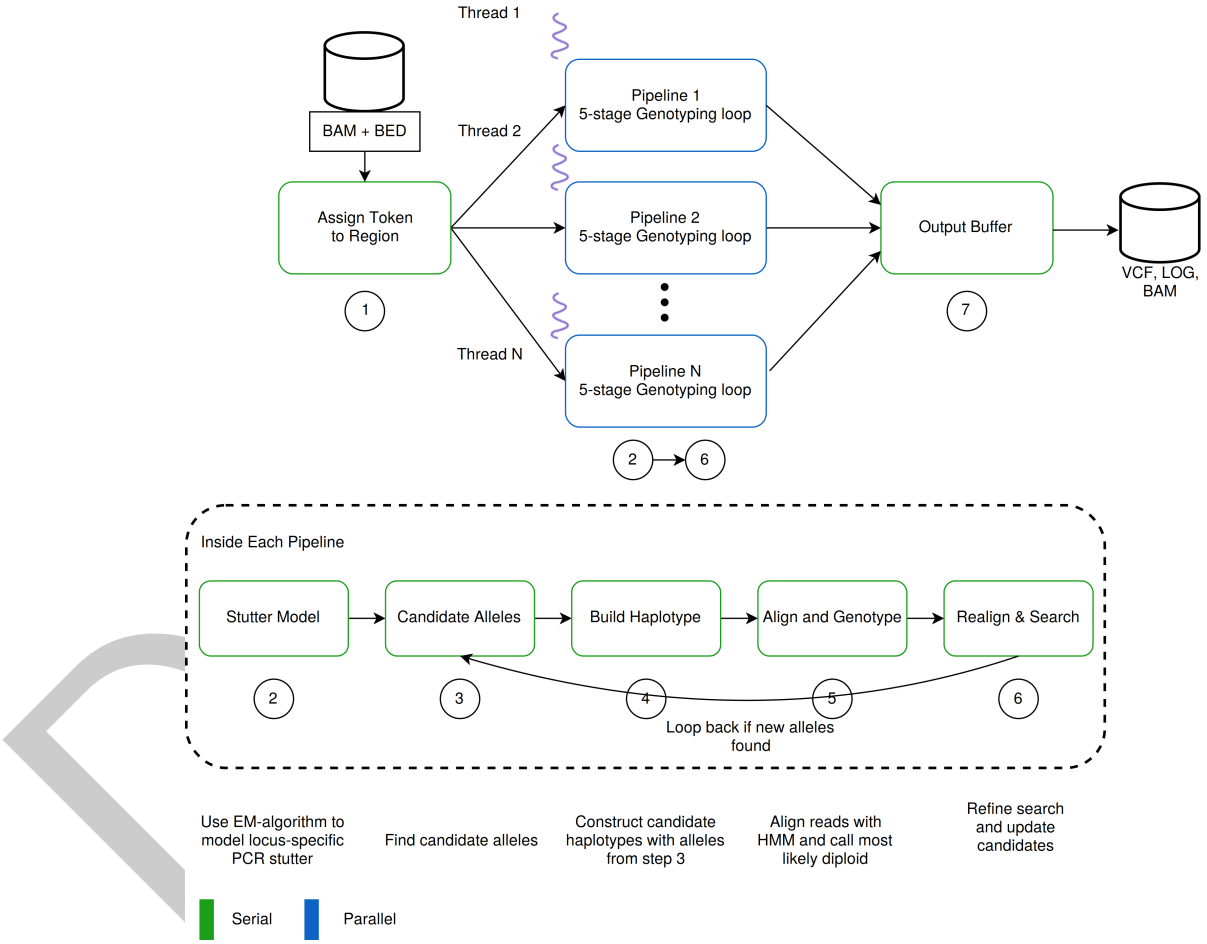


Figure 1: Overview of the HipSTR-MT genotyping pipeline. A serial dispatch stage (1) creates one token per genomic region and loads the needed chromosome. Regions are distributed across parallel worker lines (2), each performing BAM/CRAM seeking, read filtering, SNP-based phasing, haplotype construction, and HMM-based realignment. A serial output stage (3) collects results and writes VCF, log, and BAM output in the original region order regardless of worker completion order, guaranteeing output identical to the serial tool.

The Journal of Open Source Software

Software Design

The unit of work is the genomic region: regions are independent under HipSTR's model and map cleanly onto the existing per-locus loop. Rather than a parallel loop over regions, HipSTR-MT uses the three-stage Taskflow pipeline in Figure 1: serial dispatch, parallel workers, serial ordered output. The serial stages bound achievable scaling, but they let the tool emit VCF, log, and BAM records in the original region order regardless of which worker finishes first, keeping output directly comparable with the serial tool.

Treating that comparability as a hard constraint drove two trade-offs. Multiply-add contraction is disabled (compiler flag `-ffp-contract=off`), giving up vectorization headroom to keep results bit-identical across instruction sets, since silent numerical divergence would make the fork unusable as a drop-in replacement. Each worker also keeps four region contexts in flight, with independent reader and alignment state, so the work-stealing scheduler can hide I/O and memory latency; this raises peak memory from ~1.6 GB to ~8 GB at 64 threads, favoring wall-clock time on machines where cores are scarcer than RAM.

Thread safety required eliminating two pieces of shared mutable state: `StutterAlignerClass`'s scratch buffers, moved into a per-`HapAligner` workspace, and the non-reentrant Cephes `bdtr` function, now mutex-guarded. Further optimizations include a shared chromosome cache, mimalloc (Leijen et al., 2019), an htslib upgrade, and SIMD dispatch via compiler target clones — improving single-threaded performance as well. Three additive flags extend the CLI without breaking backward compatibility.

Table 1: Summary of code-level changes introduced in HipSTR-MT, grouped by category (parallelization, thread safety, memory, vectorization, dependency/I/O, and CLI flags), with the effect of each change on behavior or performance. Items are code changes unless specified as compiler flags. Full implementation detail for each entry is provided in the repository README.

Category	Change	Effect
Parallelization	Taskflow pipeline	Reads, filters, and writes regions across three concurrent stages.
	SNP-phasing safety	Mutexes protect shared phasing state across worker threads.
	Output buffering	Per-thread output buffers are flushed in original region order.
	Lock-free VCF records	Workers render VCF lines as text without holding the output lock.
Thread safety	StutterAligner buffers	Per-`HapAligner` scratch buffers eliminate a shared-state race.
	Cephes `bdtr` mutex	Guards a non-reentrant function used in allele-bias computation.
Memory	HapAligner buffer reuse	Reuses per-aligner scratch buffers across reads instead of reallocating.
	ASCII case conversion	Replaces locale-aware `toupper`/`tolower` in hot per-base loops.
	`fast_log_sum_exp` overload	Pointer-pair variant avoids a vector copy at call sites.
	mimalloc	Reduces allocator overhead from small per-read/per-locus allocations.
	Chromosome cache + eviction	Chromosomes shared across workers instead of duplicating per thread. Evicted once there are no workers using it.

DRAFT

Category	Change	Effect
Vectorization	`-flto=auto` (compiler flag)	Enables link-time optimization across translation units.
	`target_clones` dispatch	Builds one code variant per ISA; dispatches to the best at runtime.
	libmvec `exp()`	Vectorizes the exponential reduction with correctly-rounded results.
	SSE batching	Processes four elements per instruction via `vfasterexp()`.
	`-ffp-contract=off` (compiler flag)	Disables multiply-add fusion to keep output bit-identical across ISAs.
Dependency/IO	htslib 1.9 -> 1.24	Replaces byte-at-a-time FASTA reads with a block-read implementation.
	libdeflate enabled	Activates a previously unused BGZF decompression path.
CLI flags	`--threads`	Sets worker thread count; auto-detected from hardware if unset.
	`--lib-from-samp`	Assigns library name from sample name when LB tags are absent.
	`--output-hap-fields`	Adds extra FORMAT fields describing full assembled haplotypes.

Performance and correctness

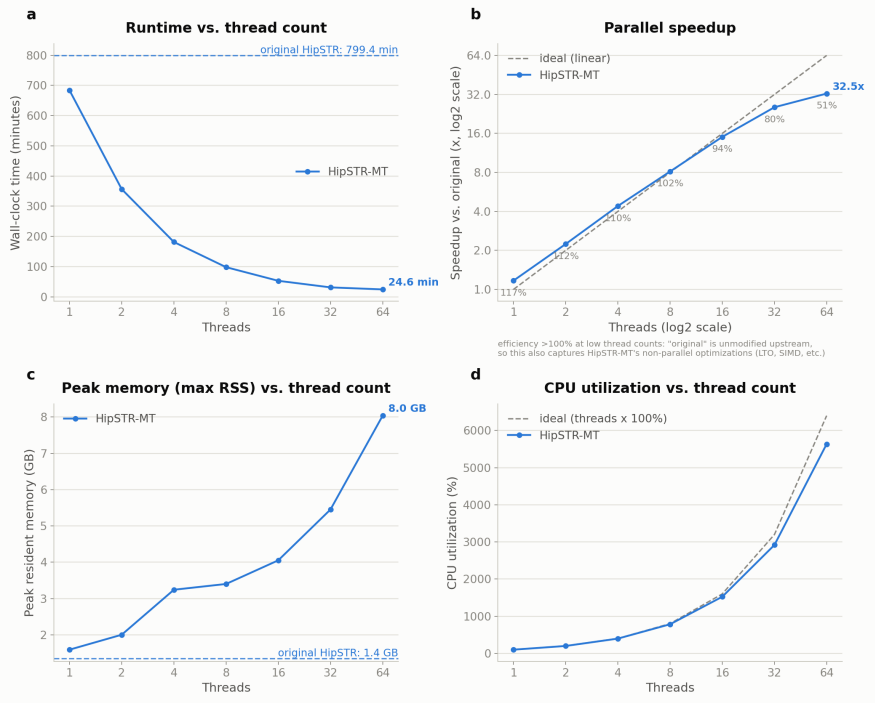


Figure 2: Performance results from processing the full NA12891 genome for all 1.5M+ STRs. a) Runtime vs. thread count. b) Parallel speedup vs. thread count, with data points annotated with scaling efficiency. c) Peak memory vs. thread count. d) CPU utilization vs. thread count.

DRAFT

Benchmarks used the NA12891 sample (accession ERR194160) against the genome-wide hg19 STR panel (1,512,240 loci):

```
./HipSTR-MT --bams ERR194160.bam --fasta hg19.fa --regions hg19_all_strs.bed \
  --str-vcf output.vcf.gz --min-reads 25 --def-stutter-model --threads N
```

with $N \in \{1, 2, 4, 8, 16, 32, 64\}$, measured via `/usr/bin/time -v` on dual Intel Xeon Silver 4216 CPUs (64 logical CPUs), Ubuntu 20.04. As Figure 2 shows, HipSTR-MT at 64 threads finishes in 24.6 minutes versus 13.3 hours for the unmodified serial baseline (a 32.5× reduction), already including the non-parallel optimizations above, so even single-threaded HipSTR-MT is measurably faster than upstream. Scaling is near-linear through 16 threads (94% efficiency), tapering to 51% by 64 threads, where threads share the machine's 32 physical cores through SMT; per-locus haplotype alignment and traceback work is small, memory-irregular, and imbalanced across loci, leaving some workers idle.

Output equivalence was verified at every thread count with a tolerant VCF comparator requiring exact genotype-call matches and allowing $\leq 10^{-3}$ relative drift in floating-point fields (GLDIFF, PDP) from SIMD/codegen-dependent summation order. All genotype records were identical across thread counts on both NA12891 and NA12892 samples. Maximum RSS grows with in-flight regions: ~8 GB at 64 threads versus ~1.6 GB at 1 thread.

Research Impact Statement

HipSTR-MT is a recent release, so its significance rests on the established user base it serves together with the performance and equivalence results above. The original HipSTR has been cited in over 330 publications since 2017 (Willems et al., 2017) and underpins large-scale STR resources. It profiled autosomal STRs in 1,916 individuals from 479 Simons Simplex Collection family quads—a mean of 1.14 million STRs per sample—for the first genome-wide SNP + STR imputation reference panel (Saini et al., 2018), and it is one of four genotypers combined by EnsembleTR into a catalog of more than 1.7 million tandem repeat loci across 3,550 individuals from the 1000 Genomes Project and H3Africa cohorts (Ziaei Jam et al., 2023). It has also been repackaged by an independent group as a graphical front end for routine forensic casework (Frontanilla et al., 2026). Because HipSTR-MT produces genotype-identical output and preserves the command-line interface apart from three additive flags, these workflows can adopt it without re-validating existing call sets.

Acknowledgements

We thank Gymrek and Turakhia lab members at UCSD for feedback and for providing the motivation and test data for this project. HipSTR-MT is a performance fork of HipSTR, originally developed by Thomas Willems and maintained by the Gymrek lab (gymrek-lab/HipSTR). Our fork is available at https://github.com/TurakhiaLab/HipSTR-MT under GPL v2, and we track upstream developments and intend to maintain it as an ongoing open-source project.

This work was supported by funding from NIH (grant U01HG013755).

AI usage disclosure

Generative AI tools were used during development and writing: Claude Code (Claude Opus 5 and Claude Sonnet 5), OpenAI Codex (gpt-5.5, gpt-5.6-terra, gpt-5.6-sol), and DeepSeek (V3). They were used to explore the original HipSTR codebase; to write test benches, the correctness-regression suite, and data-collection and debugging scripts; and to implement or assist with specific changes, including SIMD vectorization of the log-sum-exp routines, two floating-point correctness fixes (ISA-dependent multiply-add contraction under `target_clones`,

DRAFT

and a single-precision accumulator in `fast_exp_sum`), fixes to the supplementary-read filter and `--output-hap-fields`, the htslib upgrade, build-portability fixes, the CI workflow, and the Bioconda recipe. AI tools also assisted in drafting and editing the README and this paper. The human authors made all core design decisions and reviewed and validated all AI-assisted output, including through regression tests against unmodified upstream HipSTR at multiple thread counts and full-genome comparisons on NA12891 and NA12892 that show exact agreement. The authors take full responsibility for the submitted materials.

References

Ahmad, T., Schuchart, J., Al Ars, Z., Niethammer, C., Gracia, J., & Hofstee, H. P. (2022). GenMPI: Cluster scalable variant calling for short/long reads sequencing data. *bioRxiv*, 2022.04.01.486779. https://doi.org/10.1101/2022.04.01.486779

Budowle, B., & Sajantila, A. (2024). Short tandem repeats—how microsatellites became the currency of forensic genetics. *Nature Reviews Genetics*, *25*(7), 450–451. https://doi.org/10.1038/s41576-024-00721-1

Frontanilla, T. S., Sousa Ferrari, M. de, Ayala, J., & Mendes-Junior, C. T. (2026). HipSTR-UI: A cross-platform graphical interface for accessible STR genotyping from next-generation sequencing data. *Forensic Science International: Genetics*, *84*, 103456. https://doi.org/10.1016/j.fsigen.2026.103456

Huang, T.-W., Lin, D.-L., Lin, C.-X., & Lin, Y. (2022). Taskflow: A lightweight parallel and heterogeneous task graph computing system. *IEEE Transactions on Parallel and Distributed Systems*, *33*(6), 1303–1320. https://doi.org/10.1109/TPDS.2021.3104255

Leijen, D., Zorn, B., & De Moura, L. (2019). Mimalloc: Free list sharding in action. In A. W. Lin (Ed.), *Programming languages and systems* (pp. 244–265). Springer International Publishing. https://doi.org/10.1007/978-3-030-34175-6_13

Mirchandani, C. D., Shultz, A. J., Thomas, G. W. C., Smith, S. J., Baylis, M., Arnold, B., Corbett-Detig, R., Enbody, E., & Sackton, T. B. (2024). A fast, reproducible, high-throughput variant calling workflow for population genomics. *Molecular Biology and Evolution*, *41*(1), msad270. https://doi.org/10.1093/molbev/msad270

Oketch, J. W., Wain, L. V., & Hollox, E. J. (2024). A comparison of software for analysis of rare and common short tandem repeat (STR) variation using human genome sequences from clinical and population-based samples. *PLOS ONE*, *19*(4), e0300545. https://doi.org/10.1371/journal.pone.0300545

Saini, S., Mitra, I., Mousavi, N., Fotsing, S. F., & Gymrek, M. (2018). A reference haplotype panel for genome-wide imputation of short tandem repeats. *Nature Communications*, *9*(1), 4397. https://doi.org/10.1038/s41467-018-06694-0

Uguen, K., Michaud, J. L., & Génin, E. (2024). Short tandem repeats in the era of next-generation sequencing: From historical loci to population databases. *European Journal of Human Genetics*, *32*(9), 1037–1044. https://doi.org/10.1038/s41431-024-01666-z

Willems, T., Zielinski, D., Yuan, J., Gordon, A., Gymrek, M., & Erlich, Y. (2017). Genome-wide profiling of heritable and de novo STR variations. *Nature Methods*, *14*(6), 590–592. https://doi.org/10.1038/nmeth.4267

Ziaei Jam, H., Li, Y., DeVito, R., Mousavi, N., Ma, N., Lujumba, I., Adam, Y., Maksimov, M., Huang, B., Dolzhenko, E., Qiu, Y., Kakembo, F. E., Joseph, H., Onyido, B., Adeyemi, J., Bakhtiari, M., Park, J., Javadzadeh, S., Jjingo, D., … Gymrek, M. (2023). A deep population reference panel of tandem repeat variation. *Nature Communications*, *14*(1), 6711. https://doi.org/10.1038/s41467-023-42278-3

DRAFT